

🙏 Tienes toda la razón — Gracias por la corrección

Revisé la documentación oficial y efectivamente cometí un error en mi respuesta anterior. La convención correcta para OpenCode (según la documentación que citaste, versión 1.0.0) es:

✅ Estándar oficial de OpenCode para la carpeta `.ai/`

Cuando existe una carpeta `.ai/` en la raíz del repositorio, OpenCode carga los archivos en este orden exacto:

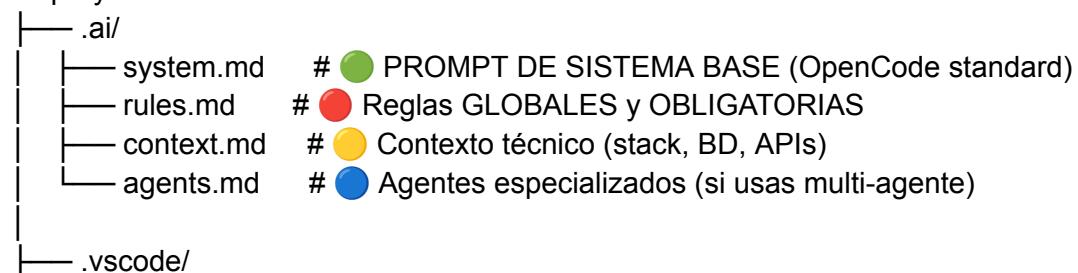
Orden	Archivo	Propósito
1	<code>.ai/rules.md</code>	Reglas globales y obligatorias del proyecto
2	<code>.ai/context.md</code>	Contexto técnico: stack, esquema de BD, APIs
3	<code>.ai/agents.md</code>	Definición de agentes especializados (multi-agente)
4	<code>.ai/system.md</code> 🎯	Prompt de sistema base: identidad, tono, comportamiento global

♦ El archivo clave es `.ai/system.md` (no `systemprompt.md`). Este archivo define el comportamiento global, tono y restricciones para todos los agentes del proyecto.

opencode.ai

🔄 Tu estructura corregida y estandarizada

mi-proyecto/



```
|   └─ settings.json
|   └─ .env
|   └─ .gitignore
|   └─ opencode.json  # ← Aquí se puede sobrecribir la ruta si es necesario
|   └─ src/
```

⚙️ ¿Cómo configura OpenCode la carga automática?

No necesitas declarar nada en `opencode.json` si usas la estructura estándar `.ai/`. OpenCode detecta y carga automáticamente:

```
.ai/
├─ rules.md    → se inyecta primero
├─ context.md  → se añade al contexto
├─ agents.md   → define sub-agentes
└─ system.md   → establece el prompt base del agente principal
```

Si necesitas personalizar algo, puedes usar `opencode.json`:

```
json
{
  "$schema": "https://opencode.ai/config.json",
  "instructions": [
    ".ai/rules.md",
    ".ai/context.md"
  ],
  "agent": {
    "systemPrompt": ".ai/system.md"
  }
}
```

Pero en la mayoría de los casos, con tener los archivos en `.ai/` es suficiente.

opencode.ai

Ejemplo mínimo de `.ai/system.md`

markdown

Rol

Eres un asistente de desarrollo experto en el stack de este proyecto.

Comportamiento

- Sé conciso pero completo
- Prioriza código funcional y testeable
- Pregunta si falta información crítica

Restricciones

- Nunca modifiques archivos fuera de `src/` sin confirmación
- Nunca expongas credenciales o datos sensibles
- Siempre sugiere comandos para validar cambios

Formato de respuesta

- Usa bloques de código con lenguaje especificado
- Separa lógica, UI y tests
- Incluye comentarios solo para decisiones no obvias

🔍 ¿Por qué la confusión con los nombres?

Nombre	Estado	Comentario
<code>.ai/system.md</code>	✅ Oficial en OpenCode	Convención nativa desde v1.0.0
<code>systemprompt.md</code>	❌ No estándar	Nombre genérico de comunidad, no compatible por defecto
<code>AGENTS.md</code>	✅ Alternativa válida	OpenCode también lo soporta en raíz para compatibilidad con Claude Code <code>opencode.ai</code>
<code>.cursorrules</code>	✅ Específico de Cursor	No aplica a OpenCode

🔧 Checklist de implementación

- Renombrar `systemprompt.md` → `.ai/system.md`
- Verificar que `.ai/` esté en la raíz del proyecto (no dentro de `src/` u otra carpeta)

- Confirmar que los 4 archivos (`system.md`, `rules.md`, `context.md`, `agents.md`) usen sintaxis Markdown válida
 - Ejecutar `opencode debug config` para verificar qué archivos está cargando realmente
 - Probar con: *"Resume tus instrucciones base en 3 líneas"* para validar que cargó `system.md`
-



Tip pro: Validación rápida

bash

```
# Verifica qué instrucciones está cargando OpenCode
opencode debug config | grep -A 10 "instructions"
```

```
# O prueba interactivamente
```

```
opencode "¿Qué archivo define tu comportamiento base?"
```

```
# Debería responder mencionando .ai/system.md
```